
Introduction to Parallel Programming

Lecture: MPI Derived Datatypes & Subcommunicators

谭光明

高性能计算机研究中心

MPI data types: why?

- Example: Root reads configuration and broadcasts it to all others.

```
// root: read configuration from  
// file into struct config  
MPI_Bcast(&cfg.nx, 1, MPI_INT, ...);  
MPI_Bcast(&cfg.ny, 1, MPI_INT, ...);  
MPI_Bcast(&cfg.du, 1, MPI_DOUBLE, ...);  
MPI_Bcast(&cfg.it, 1, MPI_INT, ...);
```

Want to do something like:

```
MPI_Bcast(  
  &cfg, 1, <type cfg>, ...);
```

```
MPI_Bcast(&cfg, sizeof(cfg),  
MPI_BYTE, ..)
```

is **not** a solution. Its not portable as no data conversion can take place

MPI is supposed to support parallel computations across heterogeneous environments and communication in such environments may require data conversions.

MPI data types: why?

- Application may need to access data noncontiguously
- For communication with a noncontiguous data layout, we can
 - Do a bunch of separate SENDs and RECVs with each contiguous piece, OR
 - Copy them into a temporary buffer and send as contiguous, OR
 - Explain the data layout to MPI and let MPI do that for you (derived datatype)

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24
25	26	27	28	29

- Example: Send column of matrix (noncontiguous in C):
 - Send each element alone?
 - Manually copy elements out into a contiguous buffer and send it?

Introduction to Datatypes in MPI

- Datatypes allow to (de)serialize arbitrary data layouts into a message stream
 - Networks provide serial channels
 - Same for block devices and I/O
- Several constructors allow arbitrary layouts
 - Recursive specification possible
 - Declarative specification of data-layout
- “what” and not “how”, leaves optimization to implementation (many unexplored possibilities!)
- Choosing the right constructors is not always simple

Making an MPI data type

- 1. Construct with
`MPI_Type_*(...)`
- 2. Commit new data type with
`MPI_Type_commit(MPI_Datatype * nt)`
- 3. After use, deallocate the data type with
`MPI_Type_free(MPI_Datatype * nt)`

**All local, non-
collective calls**

Derives type size and extent

- Get the total **size**(in bytes) of datatype in a message

```
int MPI_type_size(MPI_Datatype newtype, int *size);
```

- Get the lower bound and the **extent**(span from the first byte to the last byte) of datatype

```
int MPI_type_get_extent(MPI_Datatype newtype,  
                        MPI_Aint *lb,           } MPI type for  
                        MPI_Aint *extent);      } memory addresses or offsets
```

- MPI allows to change the extent of a datatype using

- `MPI_Type_create_resized`
- `Sizeof`
- `MPI_Get_address/MPI_Aint_diff`

size=6, extent= 8



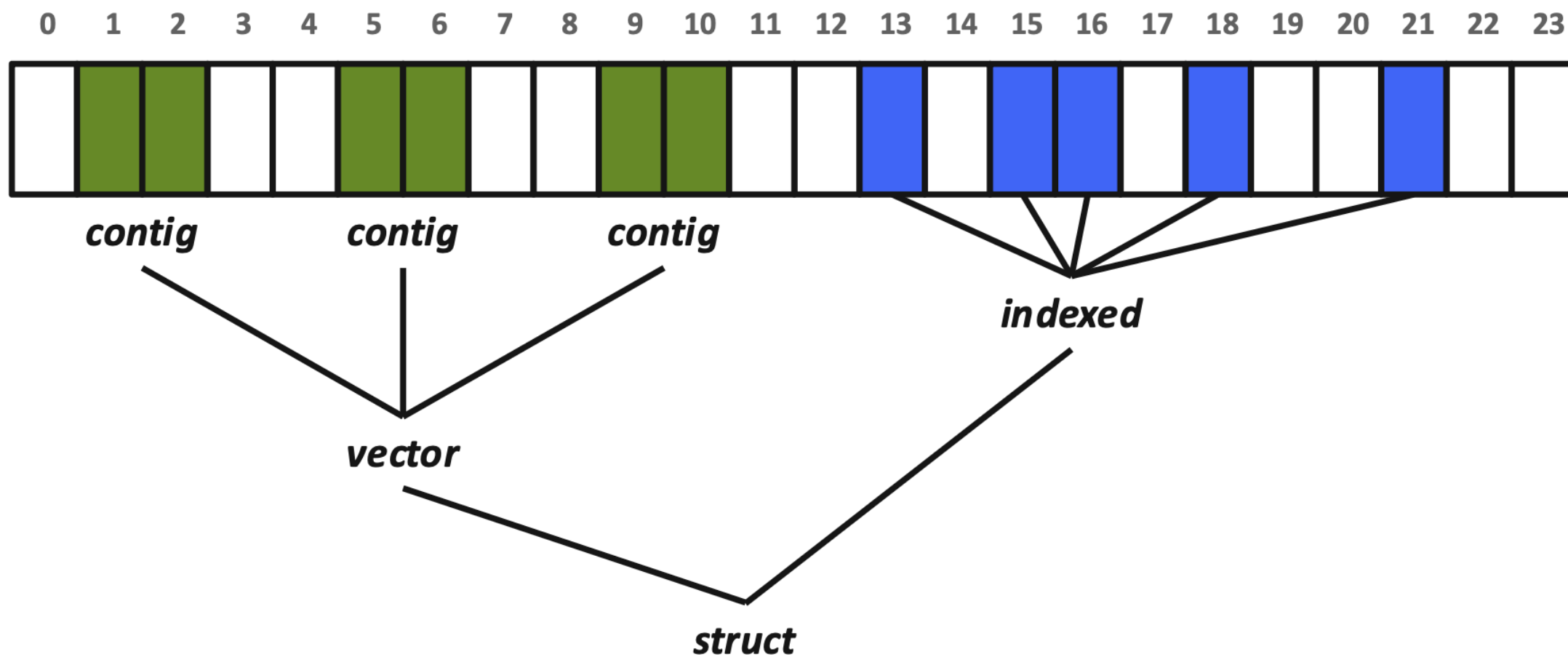
Derived Types

- For more complex or user-created datatypes, MPI provides routines to represent them as well
 - Contiguous
 - Vector/Hvector
 - Indexed/Hindexed
 - Struct
 - Subarray
- Can create complex datatype by **nesting** both basic datatypes and derived datatypes
- Datatype **created once** and **reusable** in many places
- Datatype is local to individual process, create is not collective

} Regular patterns
Popular datatypes

} Irregular patterns
Less popular datatypes

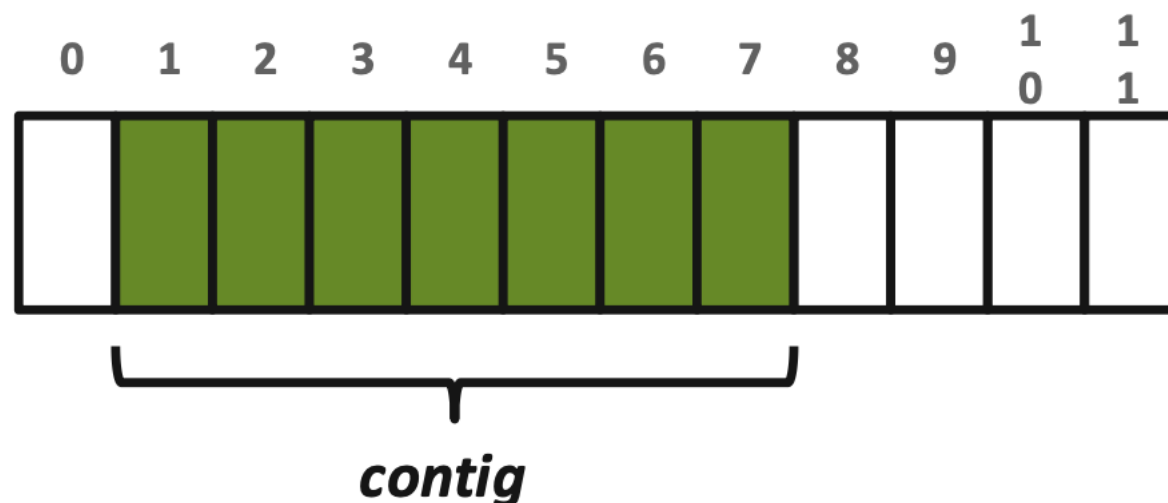
Derived Datatype Example



MPI_Type_vector

```
MPI_Type_contiguous(int count, MPI_Datatype oldtype, MPI_Datatype *newtype);
```

count 7 (no. of elements in oldtype)
oldtype MPI_INT



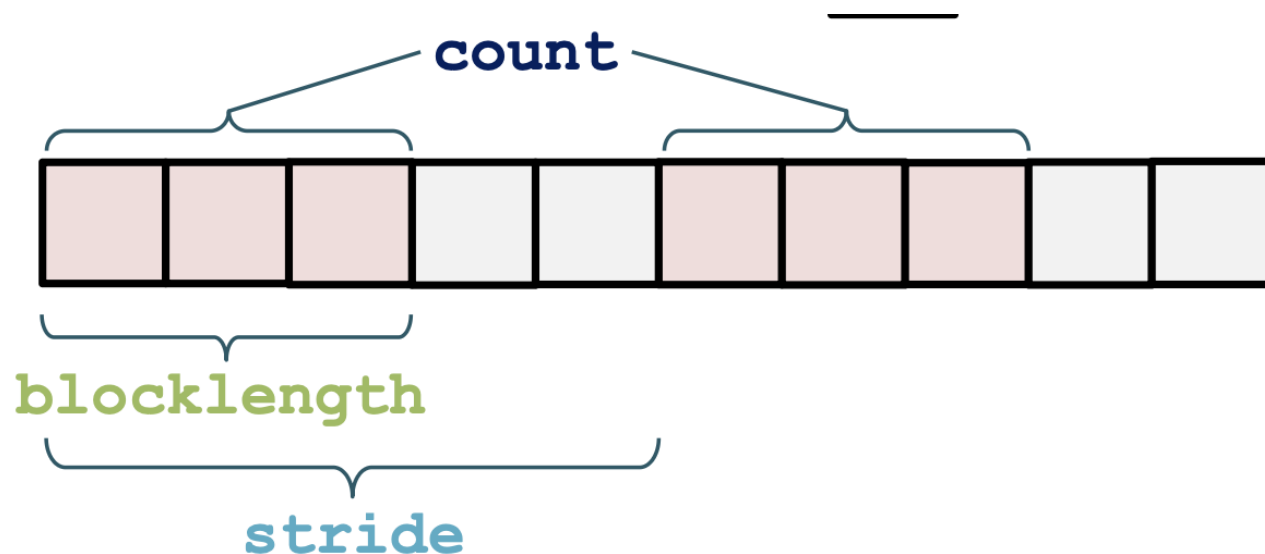
```
MPI_Datatype nt;
MPI_Type_vector(
    7, MPI_INT, &nt);

MPI_Type_commit(&nt);
// use nt...
MPI_Type_free(&nt);
```

MPI_Type_vector

```
MPI_Type_vector(int count, int blocklength, int stride,  
                MPI_Datatype oldtype, MPI_Datatype * newtype);
```

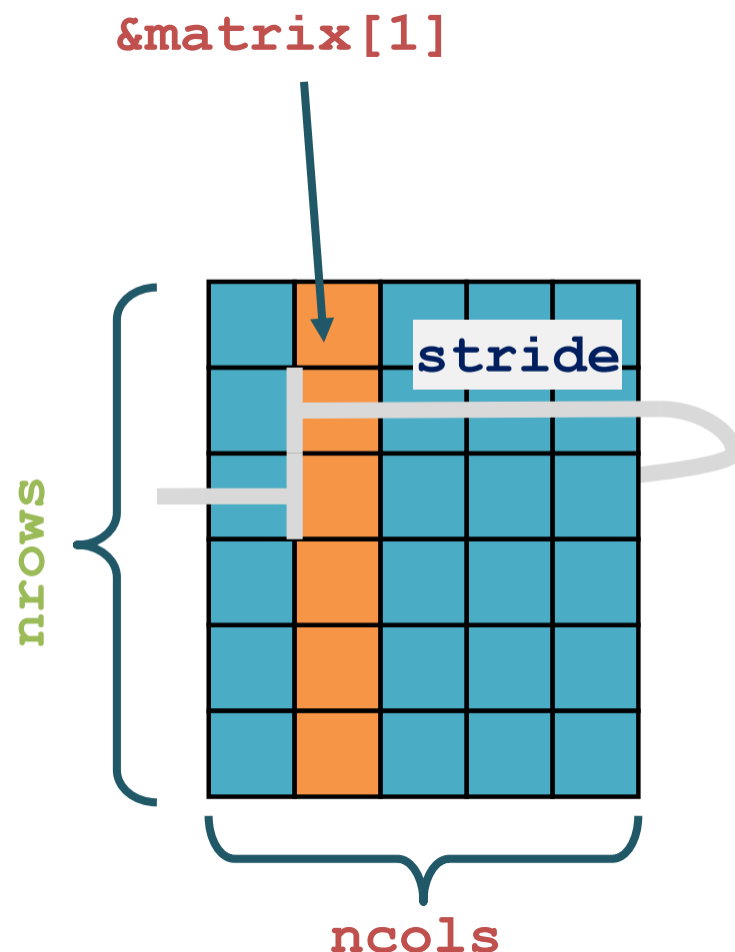
count 2 (no. of blocks)
blocklength 3 (no. of elements in each block)
stride 5 (no. of elements b/w start of each block)
oldtype **MPI_INT**



```
MPI_Datatype nt;  
MPI_Type_vector(  
    2, 3, 5, MPI_INT, &nt);  
  
MPI_Type_commit(&nt);  
// use nt..  
MPI_Type_free(&nt);
```

Sending a column of a matrix in C

- Row-major data layout in C $\rightarrow$ cannot use plain array



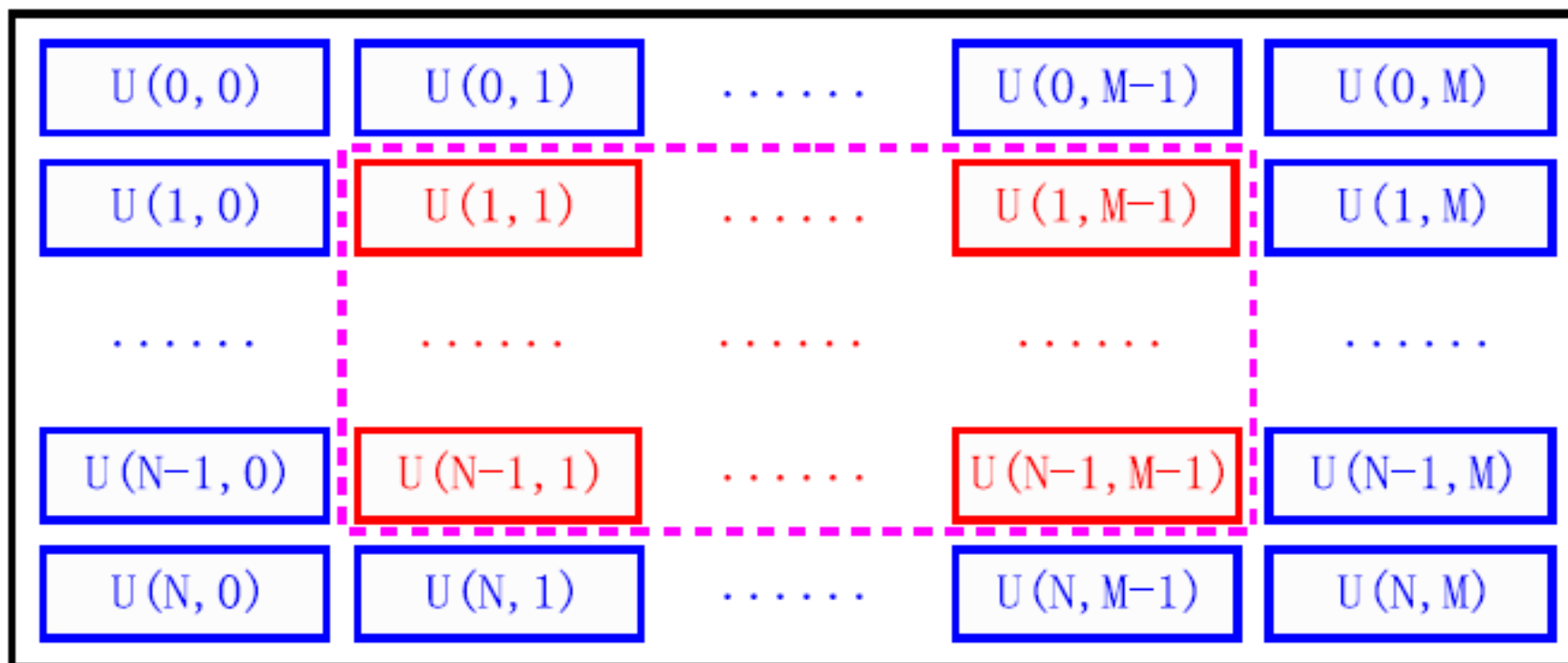
```
double matrix[30];
MPI_Datatype nt;

// count = nrows, blocklength = 1,
// stride = ncols
MPI_Type_vector(nrows, 1, ncols,
MPI_DOUBLE, &nt);
MPI_Type_commit(&nt);

// send column
MPI_Send(&matrix[1], 1, nt, ...);

MPI_Type_free(&nt);
```

Quiz

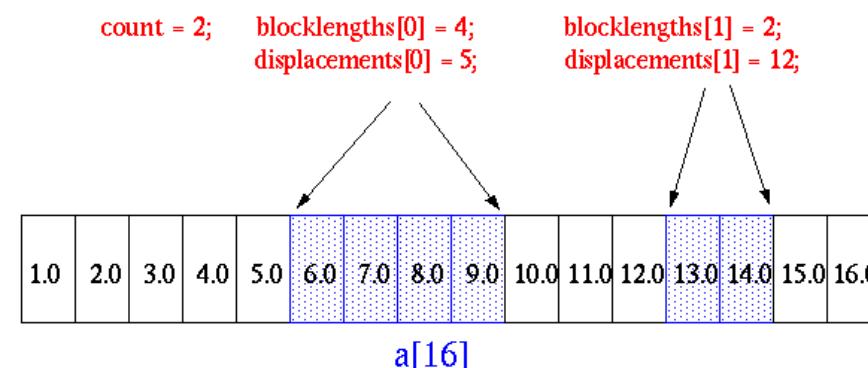


```
int a[n+1][m+1]; MPI_Datatype N_T;  
  
MPI_Type_vector(n-1,m-1,m+1,MPI_INT,&N_T);  
MPI_Type_commit(&N_T);  
MPI_SEND(&(a[1][1]),1,N_T, right, tag, comm);
```

MPI_Type_indexed

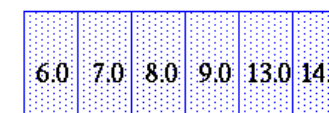
```
MPI_MPI_Type_indexed(int count, int blocklens, int indices[],  
MPI_Datatype oldtype, MPI_Datatype *newtype)
```

- **count** number of blocks -- also number of entries in indices and blocklens
- **blocklens** number of elements in each block
- **indices** displacement of each block in multiples of old_type



```
MPI_Type_indexed(count, blocklengths, displacements, MPI_FLOAT, &indextype);
```

```
MPI_Send(&a, 1, indextype, dest, tag, comm);
```



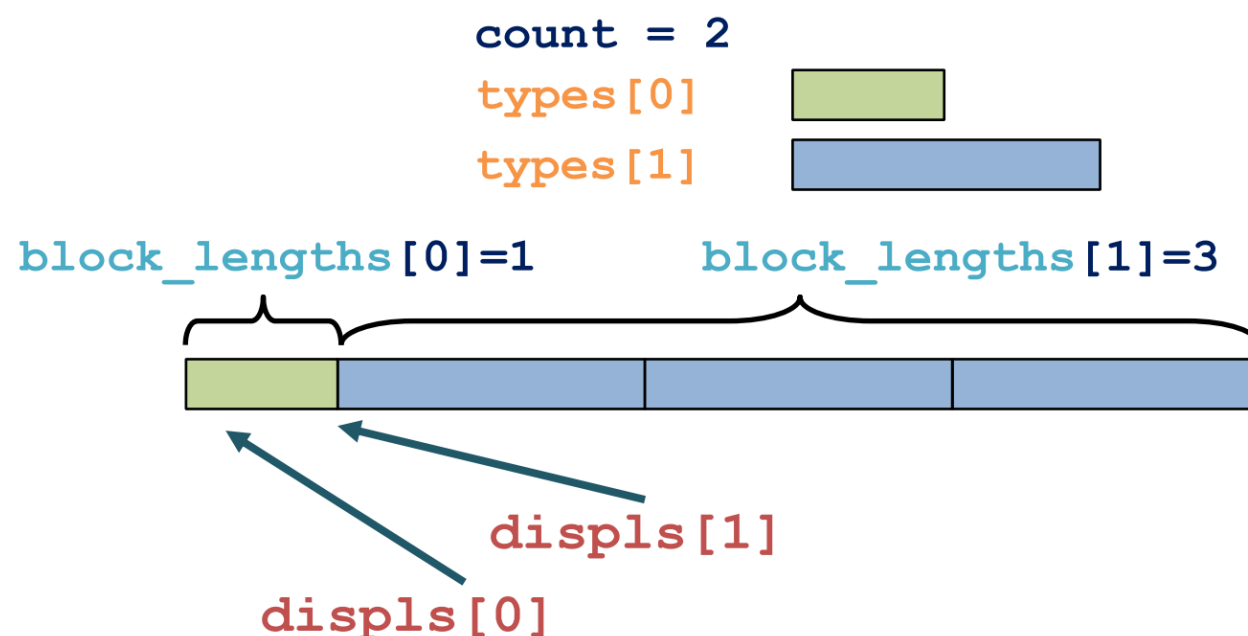
1 element of
indextype

MPI_Type_create_struct

- Describe blocks with arbitrary data types and arbitrary displacements

```
MPI_Type_create_struct(int count, int block_lengths[],
MPI_Aint displs[], MPI_Datatype types[], MPI_Datatype * newtype);
```

The contents of **displs** are either the displacements in **bytes** of the block bases or **MPI addresses**



```
typedef struct { float x,y,z,velocity; int n,type; } Particle;
Particle particles[NELEM];
```

```
MPI_Type_extent(MPI_FLOAT,&extent);
```

```
count = 2; oldtypes[0] = MPI_FLOAT; oldtypes[1] = MPI_INT;
offsets[0] = 0; offsets[1] = 4 * extent;
blockcounts[0] = 4; blockcounts[1] = 2;
```



```
MPI_Type_struct(count, blockcounts, offsets, oldtypes, &particletype);
```

```
MPI_Send(particles, NELEM, particletype, dest, tag, comm);
```

Sends entire (NELEM) array of particles, each particle being comprised four floats and two integers.

A sub-array type: MPI_Type_create_subarray

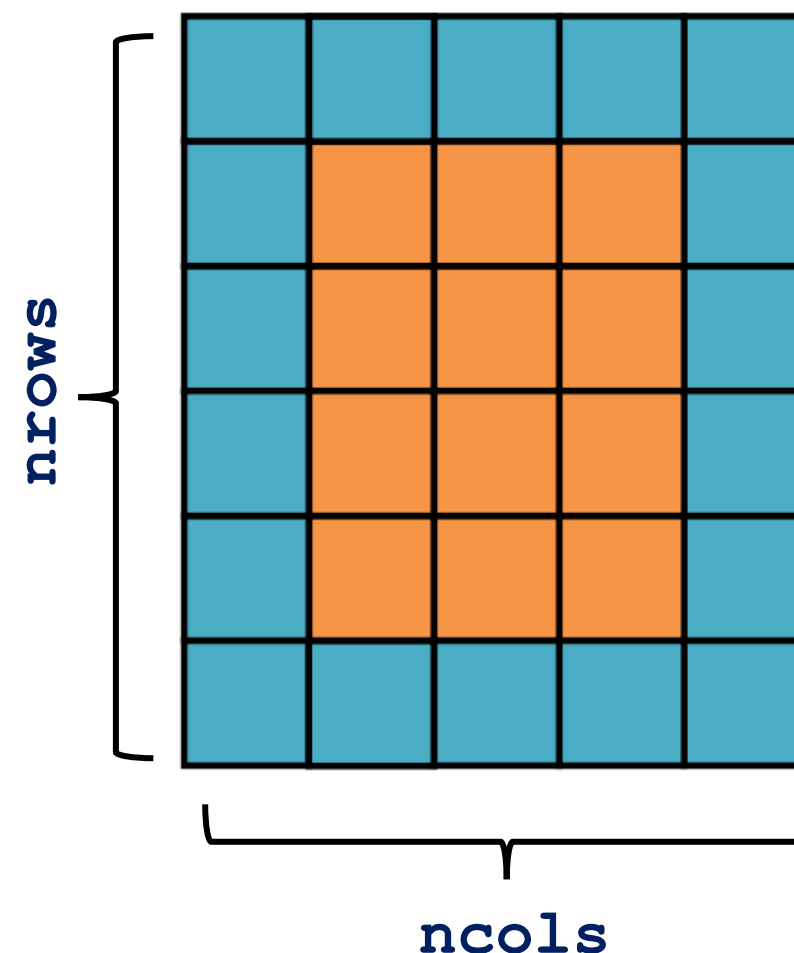
```
MPI_Type_create_subarray(int dims,  
int ar_sizes[], int ar_subsizes[], int ar_starts[],  
int order, MPI_Datatype oldtype, MPI_Datatype * newtype
```

- **dims:** dimension of the array
- **ar_sizes:** array with sizes of array (dims entries)
- **ar_subsizes:** array with sizes of subarray (dims entries)
- **ar_starts:** start indices of the subarray inside array (dims entries), start at 0 (also in Fortran)
- **order**
 - row-major: MPI_ORDER_C
 - column-major: MPI_ORDER_FORTRAN

Example for a sub-array type: “bulk” of a matrix

```
dims          2
ar_sizes      {ncols, nrows}
ar_subsizes   {ncols-2, nrows-2}
ar_starts     {1, 1}
order        MPI_ORDER_C
oldtype       MPI_INT
```

```
MPI_Type_create_subarray(dims,
ar_sizes, ar_subsizes,
ar_starts, order, oldtype, &nt);
MPI_Type_commit(&nt);
// use nt...
MPI_Send(&buf[0], 1, nt, ...); // etc.
MPI_Type_free(&nt);
```



How to obtain and handle addresses?

```
MPI_Get_address(const void *location, MPI_Aint *address);  
MPI_Aint MPI_Aint_diff(MPI_Aint addr1, MPI_Aint addr2);  
MPI_Aint MPI_Aint_add(MPI_Aint base, MPI_Aint disp);
```

■ Example:

```
double a[100];  
MPI_Aint a1, a2, disp;  
MPI_Get_address(&a[0], &a1);  
MPI_Get_address(&a[50], &a2);  
disp = MPI_Aint_diff(a2, a1);
```

■ Result would usually be **disp = 400** (50 × 8)

Datatype Selection Order

- Simple and effective performance model:
 - More parameters == slower
- predefined < contig < vector < index < struct
- Some (most) MPIs are inconsistent
 - But this rule is portable
- Advice to users:
 - Try datatype "compression" bottom-up

Summary

- A flexible tool to communicate complex data structures in MPI
- Most important calls:

```
MPI_Type_vector (second simplest)
MPI_Type_create_subarray
MPI_Type_create_struct (most advanced)
MPI_Type_commit/MPI_Type_free
MPI_Get_address,
MPI_Aint_add, MPI_Aint_diff
MPI_Type_get_extent, MPI_Type_size
MPI_Type_create_resized
```

- Other useful features:
 - MPI_Type_contiguous, MPI_Type_indexed,...
- Matching rule: send and receive match if specified basic
- datatypes match one by one, regardless of displacements
 - Correct displacements at receiver side are automatically matched to the corresponding data items

Summary

- Derived datatypes are a sophisticated mechanism to describe ANY layout in memory
 - Hierarchical construction of derived datatypes allows them to be just as complex as the data layout is
 - More complex layouts require more complex datatype constructions
- Current state of MPI implementations might be a bit lagging in performance, but it is improving
 - Increasing amount of hardware support to process derived datatypes on the network hardware
 - If the performance is lagging when you try it out, complain to the MPI implementer, don't just stop using it!

Quiz

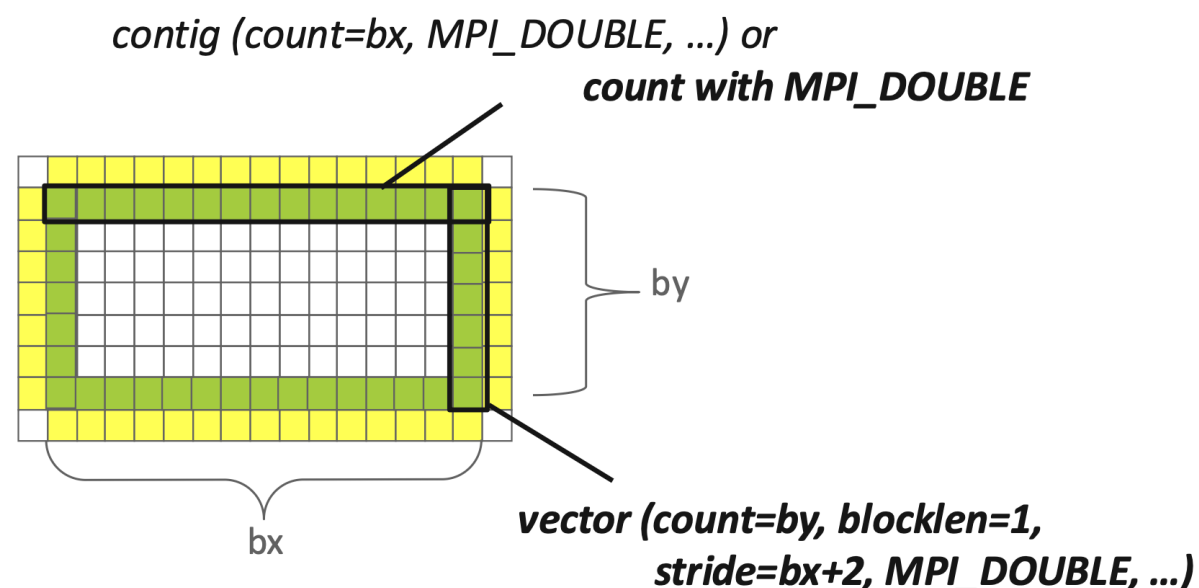
- Which one is a use case for MPI derived datatypes?
 - a. Noncontiguous elements of an array
 - b. A set of members in a struct (in C) or type (in Fortran)
 - Answer: both a and b

- Which of the bindings `MPI_Type_*` is most flexible?
 - Answer: `MPI_Type_create_struct`

- The use of MPI derived datatype is **helpful** but can be **unsafe**
 - Answer: correct!

Exercise: Stencil with Derived Datatypes

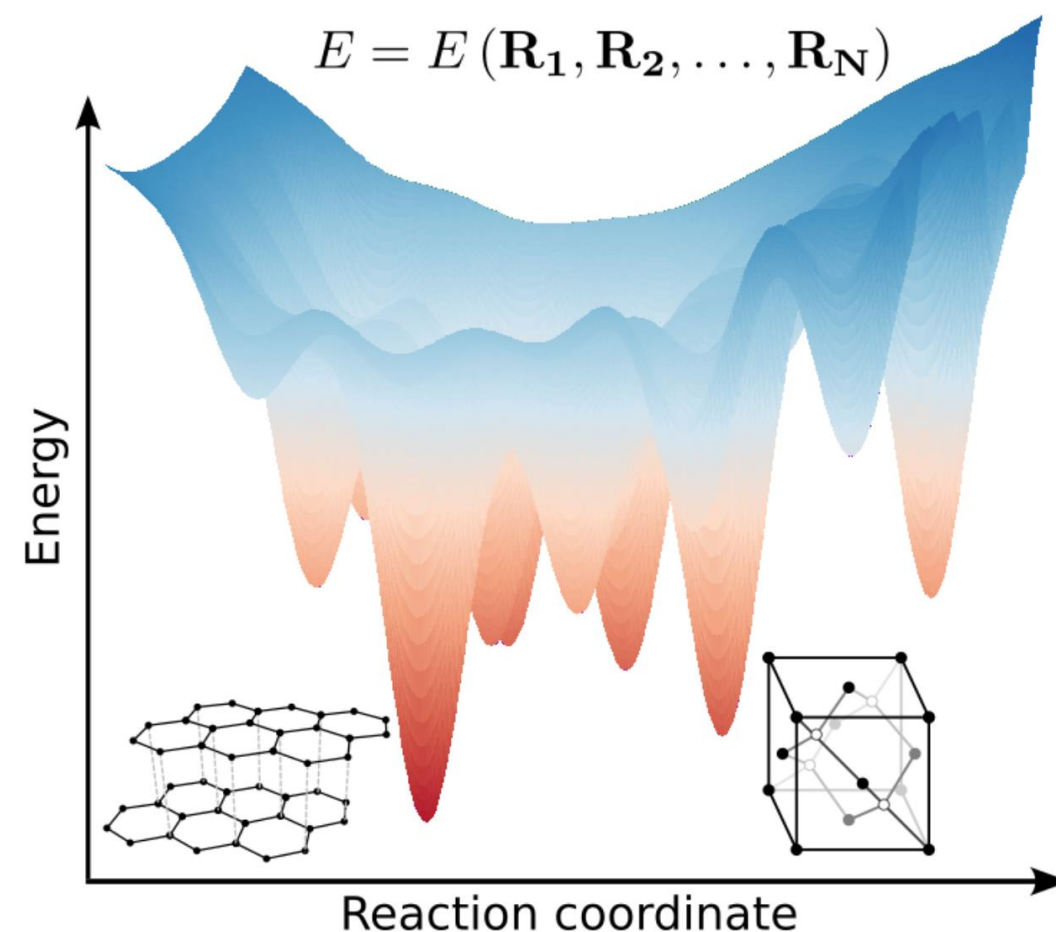
- In the basic version of the stencil code
 - Used nonblocking communication
 - Used manual packing/unpacking of data
- Let's try to use derived datatypes
 - Specify the locations of the data instead of manually packing/unpacking



Using a larger number of walkers?

- Global optimization methods may require substantial progress in sequential exploration as well
 - Accelerating the search via increasing the number of walkers stagnates at some point and will not boost the discovery of unexplored thermodynamically stable minima
- Energy/forces obtained from higher level of theory are computational very demanding

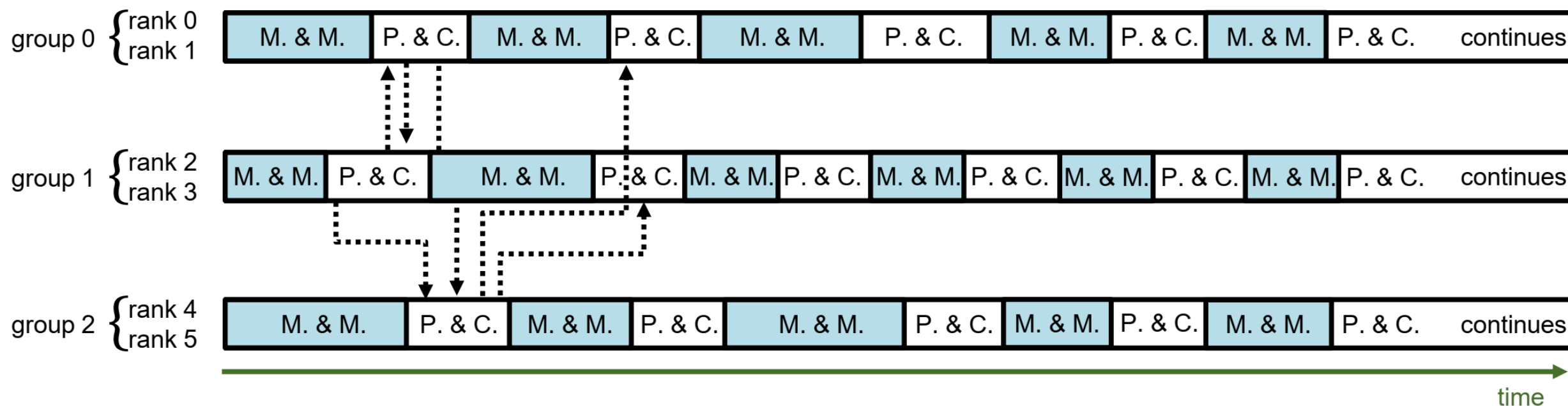
Success rate is elevated if
each walker runs faster
rather than expanding the walkers!



Multiple processes serving one walker

- Creating groups of processes and assigning each walker to a group
 - Each group should preferably have its own communicator
 - Collective communications takes place within each of the new communicators

Example of searching new atomistic structures:
3 groups each containing 2 processes



MPI Subcommunicators: use cases

- Let's assume you have 100 3D FFTs on a large mesh and you want to run on 1000 cores:
 - Each FFT on all 1000 cores, so consecutive runs:
 - Each FFT on one core, then 900 cores remain idle:
- MPMD: Multiple Program Multiple Data Execution Model
 - Different programs can be run simultaneously: communicating via MPI but then using `MPI_COMM_WORLD` should be avoided in collective communications.
- Sampling algorithms with multiple walkers modeling diffusion processes in physics and chemistry:
 - Many walkers to maximize the randomness, however, it is also required that a walker advances as fast as possible, for example kinetic Monte Carlo
 - Then, each walker on one core not a good choice

Groups and Communicators

- An MPI group is an ordered collection of processes
- Each process inside a group has a unique rank
- A new intracommunicator can be derived from a group, effectively enabling communication (point-to-point or collective) that is restricted to this group
- Predefined intracommunicators:
 - `MPI_COMM_WORLD`
 - `MPI_COMM_SELF` (contains only the process itself)
- Two possible scenarios:
 - Create a group containing subsets of the processes in a communicator and then creating a communicator from that group.
 - Directly creating a subcommunicator from a communicator

Handling Groups

- Important group handling subroutines
- Construct group from existing communicator (COMM):
 - `MPI_Comm_group(MPI_Comm comm, MPI_Group *group)`
- Generate new group by including ranks from existing group:
 - `MPI_Group_incl(MPI_Group group, int n, int *ranks[], MPI_Group *newgroup)`
- Generate new group by excluding ranks from existing group:
 - `MPI_Group_excl(MPI_Group group, int n, int *ranks, MPI_Group *newgroup)`
- Destroy group:
 - `MPI_Group_free(MPI_Group *group)`

Creating an Intracommunicator

- A communicator can be derived from an existing group (collective):

```
MPI_Comm_create(MPI_Comm comm, MPI_Group group, MPI_Comm *newcomm)
```

- Deallocating the communicator object:

```
MPI_Comm_free(MPI_Comm *comm)
```

```
#include "mpi.h"
MPI_Comm comm_world, comm_worker;
MPI_Group group_world, group_worker;
int ierr;

comm_world = MPI_COMM_WORLD;
MPI_Comm_group(comm_world, &group_world);
MPI_Group_excl(group_world, 1, 0, &group_worker); /* process 0 not member */

MPI_Comm_create(comm_world, group_worker, &comm_worker);
```

Example

```
MPI_Comm_group(MPI_COMM_WORLD, &group_w);
if(irank_w < 6) {
    if(irank_w % 2 == 0) {irank_list[0] = irank_w; irank_list[1] = irank_w + 1;}
    if(irank_w % 2 == 1) {irank_list[0] = irank_w - 1; irank_list[1] = irank_w;}
    MPI_Group_incl(group_w, 2, irank_list, &group_new);
}
else {
    group_new = MPI_GROUP_EMPTY;
}
MPI_Comm_create(MPI_COMM_WORLD, group_new, &comm_new);
if(comm_new != MPI_COMM_NULL) {
    MPI_Comm_rank(comm_new, &irank_l);
    MPI_Comm_size(comm_new, &nrank_l);
    printf("irank_w, nrank_w = %4d %4d and\n", irank_w, nrank_w, irank_l, nrank_l);
}
if(group_new != MPI_GROUP_EMPTY) MPI_Group_free(&group_new);
if(comm_new != MPI_COMM_NULL) MPI_Comm_free(&comm_new);
MPI_Group_free(&group_w);
...
```

Example

Running on 6 processes:

```
mpirun -n 6 ./a.out |sort -n -k2
irank_w,nrank_w= 0 6 and irank_l,nrank_l= 0 2
irank_w,nrank_w= 1 6 and irank_l,nrank_l= 1 2
irank_w,nrank_w= 2 6 and irank_l,nrank_l= 0 2
irank_w,nrank_w= 3 6 and irank_l,nrank_l= 1 2
irank_w,nrank_w= 4 6 and irank_l,nrank_l= 0 2
irank_w,nrank_w= 5 6 and irank_l,nrank_l= 1 2
```

Running on 7 processes:

```
mpirun -n 7 ./a.out |sort -n -k2
irank_w,nrank_w= 0 7 and irank_l,nrank_l= 0 2
irank_w,nrank_w= 1 7 and irank_l,nrank_l= 1 2
irank_w,nrank_w= 2 7 and irank_l,nrank_l= 0 2
irank_w,nrank_w= 3 7 and irank_l,nrank_l= 1 2
irank_w,nrank_w= 4 7 and irank_l,nrank_l= 0 2
irank_w,nrank_w= 5 7 and irank_l,nrank_l= 1 2
```

Direct creation of a communicator

- Creating a new communicator based on color and key codes:

```
MPI_Comm_split(MPI_Comm comm,int color,int key,MPI_Comm *newcomm)
```

- Collective operation:
 - If color is set to `MPI_UNDEFINED`, `newcomm` returns `MPI_COMM_NULL`
- color: controls the assignment of processes in the new subset
 - Nonnegative integer
 - Processes with the same value of color in the same subset
- key: controls the rank assignment in the new communicator
 - For every pair of processes:
 - process with a smaller value of key results in a smaller value of rank in `newcomm`
 - in case of identical key values, the order of ranks follows the order in the parent one

Example

Iam	0	1	2	3	4	5
irow	0	0	1	1	2	2
jcol	0	1	0	1	0	1

(0)	(1)
(0)	(1)
(2)	(3)
(2)	(3)
(4)	(5)
(4)	(5)

(0)	(1)
(0)	(1)
(2)	(3)
(0)	(1)
(4)	(5)
(0)	(1)

(0)	(1)
(0)	(0)
(2)	(3)
(1)	(1)
(4)	(5)
(2)	(2)

```

/* logical 2D topology with nrow rows and mcol columns */
irow = Iam/mcol;          /* logical row number */
jcol = mod(Iam, mcol);    /* logical column number */
comm2D = MPI_COMM_WORLD;

MPI_Comm_split(comm2D, irow, jcol, row_comm);
MPI_Comm_split(comm2D, jcol, irow, col_comm);

```

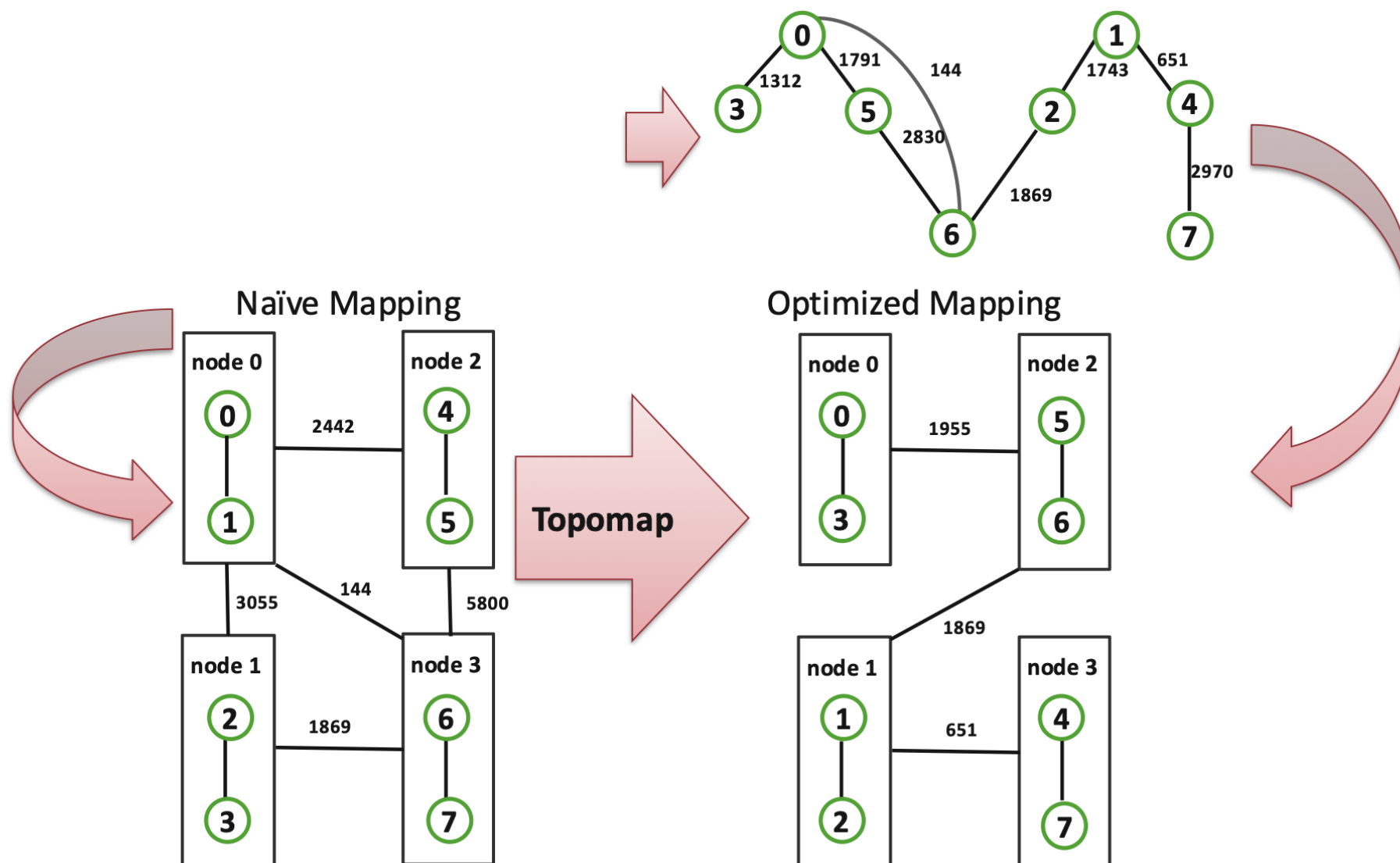
Topology Mapping

- Topology mapping basics
 - Allocation mapping vs. rank reordering
 - Ad-hoc solutions vs. portability
- MPI topologies
 - Cartesian
 - Distributed graph

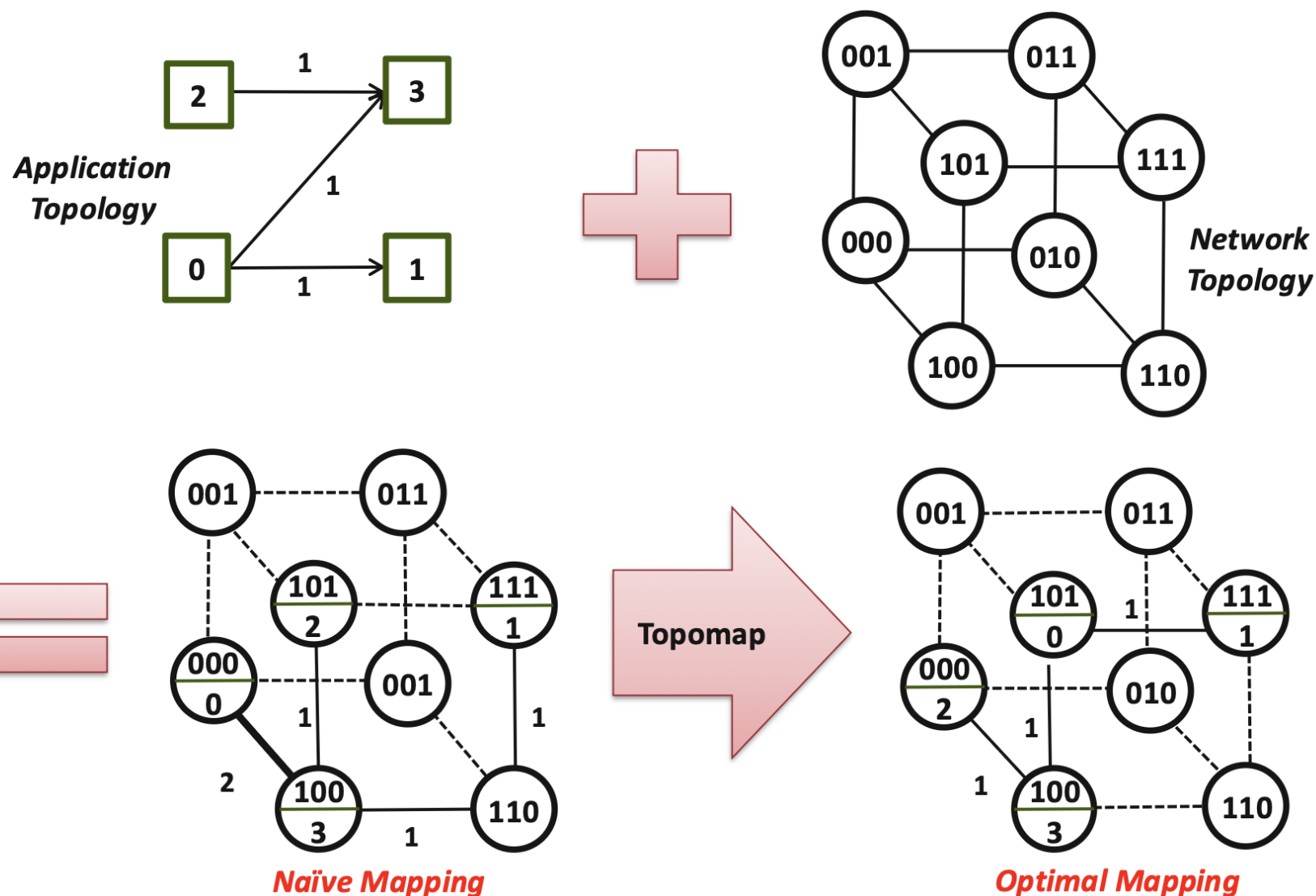
Topology Mapping Basics

- MPI supports rank reordering
 - Change numbering in a given allocation to reduce congestion or dilation
 - Sometimes automatic (early IBM SP machines)
- Properties
 - Always possible, but effect may be limited (e.g., in a bad allocation)
 - Portable way: MPI process topologies
 - Network topology is not exposed
 - Manual data shuffling after remapping step

Example: On-Node Reordering



Off-Node (Network) Reordering



MPI Topology Intro

- Convenience functions (in MPI-1)
 - Create a graph and query it, nothing else
 - Useful especially for Cartesian topologies
 - Query neighbors in n-dimensional space
 - Graph topology: each rank specifies full graph 
- Scalable Graph topology (MPI-2.2)
 - Graph topology: each rank specifies its neighbors or an arbitrary subset of the graph
- Neighborhood collectives (MPI-3.0)
 - Adding communication functions defined on graph topologies (neighborhood of distance one)

MPI_Cart_create

```
MPI_Cart_create(MPI_Comm comm_old, int ndims, const int *dims, const int *periods, int reorder, MPI_Comm *comm_cart)
```

- Specify ndims-dimensional topology
 - Optionally periodic in each dimension (Torus)
- Some processes may return `MPI_COMM_NULL`
 - Product sum of dims must be $\leq P$
- Reorder argument allows for topology mapping
 - Each calling process may have a new rank in the created communicator
 - Data has to be remapped manually

MPI_Cart_create Example

```
#include "mpi.h"
MPI_Comm old_comm, new_comm;
int ndims, reorder, periods[2], dim_size[2];

old_comm = MPI_COMM_WORLD;
ndims = 2;           /* 2D matrix/grid */
dim_size[0] = 3;     /* rows */
dim_size[1] = 2;     /* columns */
periods[0] = 1;      /* row periodic (each column forms a ring) */
periods[1] = 0;      /* columns nonperiodic */
reorder = 1;         /* allows processes reordered for efficiency */

MPI_Cart_create(old_comm, ndims, dim_size, periods, reorder, &new_comm);
```

0,0 (0)	0,1 (1)
1,0 (2)	1,1 (3)
2,0 (4)	2,1 (5)

periods(0)=.true.
periods(1)=.false.

	-1,0 (4)	-1,1 (5)	
0, -1 (-1)	0,0 (0)	0,1 (1)	0,2 (-1)
1, -1 (-1)	1,0 (2)	1,1 (3)	1,2 (-1)
2, -1 (-1)	2,0 (4)	2,1 (5)	2,2 (-1)
	3,0 (0)	3,1 (1)	

periods(0)=.false.
periods(1)=.true.

	-1,0 (-1)	-1,1 (-1)	
0, -1 (1)	0,0 (0)	0,1 (1)	0,2 (0)
1, -1 (3)	1,0 (2)	1,1 (3)	1,2 (2)
2, -1 (5)	2,0 (4)	2,1 (5)	2,2 (4)
	3,0 (-1)	3,1 (-1)	

MPI_Dims_create

```
MPI_Dims_create(int nnodes, int ndims, int *dims)
```

- Create dims array for Cart_create with nnodes and ndims
 - Dimensions are as close as possible (well, in theory)
- Non-zero entries in dims will not be changed
 - nnodes must be multiple of all non-zeroes
- Makes life a little bit easier
 - Some problems may be better with a non-square layout though

```
int p;  
MPI_Comm_size(MPI_COMM_WORLD, &p);  
MPI_Dims_create(p, 3, dims);  
  
int periods[3] = {1,1,1};  
MPI_Comm topocomm;  
MPI_Cart_create(comm, 3, dims, periods, 0, &topocomm);
```

Cartesian Query Functions

- Library support and convenience!
- `MPI_Cartdim_get()`
 - Gets dimensions of a Cartesian communicator
- `MPI_Cart_get()`
 - Gets size of dimensions
- `MPI_Cart_rank()`
 - Translate coordinates to rank
- `MPI_Cart_coords()`
 - Translate rank to coordinates

Cartesian Communication Helpers

```
MPI_Cart_shift(MPI_Comm comm, int direction, int disp,  
               int *rank_source, int *rank_dest)
```

- Shift in one dimension
 - Dimensions are numbered from 0 to ndims-1
 - Displacement indicates neighbor distance (-1, 1, ...)
 - May return MPI_PROC_NULL
- Very convenient, all you need for nearest neighbor communication
 - No "over the edge" though

Cartesian Communication Helpers

```
/* create Cartesian topology for processes */
dims[0] = nrow;      /* number of rows      */
dims[1] = mcol;      /* number of columns   */
period[0] = 1;       /* cyclic in this direction */
period[1] = 0;       /* no cyclic in this direction */
MPI_Cart_create(MPI_COMM_WORLD, ndim, dims, period, reorder, &comm2D);
MPI_Comm_rank(comm2D, &me);
MPI_Cart_coords(comm2D, me, ndim, coords);

index = 0;          /* shift along the 1st index (out of 2) */
displ = 1;          /* shift by 1 */
MPI_Cart_shift(comm2D, index, displ, &source, &dest1);
```

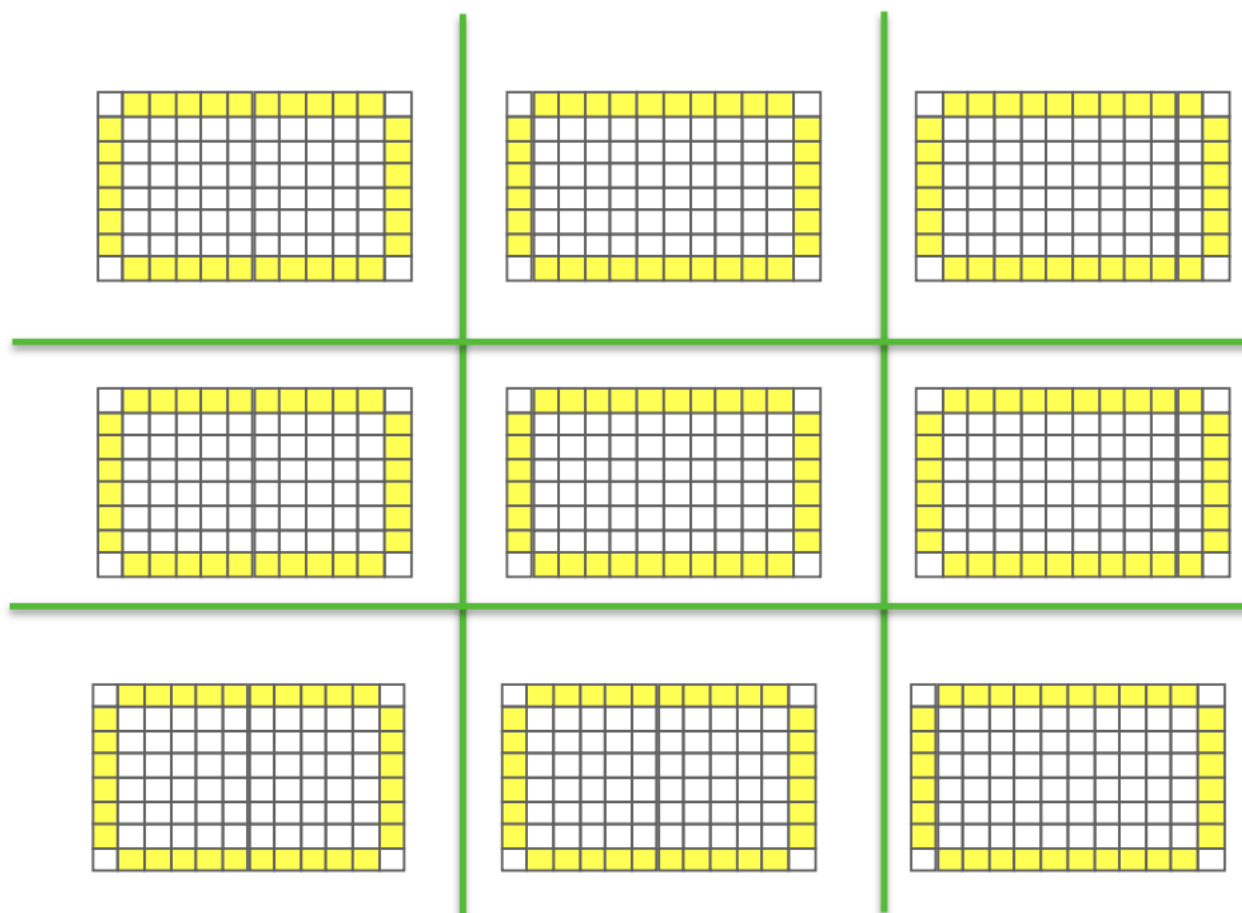
<i>0,0</i> <i>(0)</i>	<i>0,1</i> <i>(1)</i>
<i>1,0</i> <i>(2)</i>	<i>1,1</i> <i>(3)</i>
<i>2,0</i> <i>(4)</i>	<i>2,1</i> <i>(5)</i>

Rank 2: source 0, destination 4

Rank 1: source 5, destination 3

Exercise: Stencil with Cartesian Topology

- `topology/stencil_carttopo.c`
- Adds calculation of neighbors with topology



Section Summary

- MPI does not expose information about the network topology (would be very complex)
- Topology functions allow users to specify application communication patterns/topology
 - Convenience functions (e.g., Cartesian)
 - Storing neighborhood relations (Graph)
- Neighborhood collectives allow user virtual topologies to be exploited in collective communication

Quiz

- Subcommunicators are suitable only for use in libraries?
 - Answer: Incorrect

- The `MPI_Comm_split` binding creates subcommunicators with disjoint groups?
 - Answer: correct

- Two groups can have common members but two communicators cannot?
 - Answer: Incorrect